

Звіт про виконання лабораторної роботи №5

Реєстрація, аутентифікація та авторизація в ASP.NET Core Identity.

Виконав: студент групи 201-пТК

Ткаченко Микола Віталійович

Запитання	Відповідь
IdentityUser та IdentityDbContext	<code>IdentityUser</code> містить вбудовані властивості (UserName, Email, PasswordHash). Розширення дозволяє додати кастомні поля. <code>IdentityDbContext</code> розширює звичайний DbContext для створення таблиць, потрібних системі Identity (напр.,AspNetUsers,AspNetRoles).
JwtBearer vs Cookie	JWT (JSON Web Token) зберігається на клієнті і надсилається в заголовку Authorization, що ідеально підходить для REST API та мобільних додатків (stateless). Cookie зберігається браузером і керується сервером, частіше використовується в MVC додатках (stateful).

Класи для обробки JWT	Використовується <code>JwtSecurityTokenHandler`</code> для генерації токена та <code>TokenValidationParameters`</code> у <code>Program.cs`</code> для його валідації.
-----------------------	---

1. Опис реалізації

- **ASP.NET Core Identity:** Підключено у проєкті `Infrastructure``. Створено клас `ApplicationUser``, який наслідується від `IdentityUser``.
- **DbContext:** `SmartHomeIdentityContext`` тепер наслідується від `IdentityDbContext``.
- **JWT Токени:** Налаштовано генерацію та валідацію JWT (Bearer) tokenів у контролері `AccountController`` (ендпоінти `/register`` та `/login``).
- **Рольова модель:** Створено контролер `SecureDevicesController``, що демонструє використання атрибутів `[Authorize]`` та `[Authorize(Roles="Admin")]``.

2. Відповіді на контрольні запитання

4. Роль UserManager і RoleManager	<code>UserManager`</code> відповідає за створення користувачів, хешування паролів та перевірку логінів. <code>RoleManager`</code> керує ролями (створення, видалення). Обидва інжекються через DI.
5. Створення JWT	Для створення токена формується список <code>Claim`</code> (ідентифікатори користувача, ролі), створюється <code>SymmetricSecurityKey`</code> і підписується за допомогою алгоритму HmacSha256.
6. Налаштування Middleware	У <code>Program.cs`</code> викликається <code>AddAuthentication(JwtBearerDefaults...)`</code> та <code>AddJwtBearer(...)`. У пайплайні обов'язково додаються <code>app.UseAuthentication()</code> перед <code>app.UseAuthorization()</code>.</code>
7. Заголовок Authorization	Клієнт надсилає заголовок <code>Authorization: Bearer `</code> . Сервер витягує токен, перевіряє підпис (Secret Key), термін дії та issuer.
8. Захищені та загальнодоступні endpoints	<code>/api/account/login`</code> та <code>/register`</code> залишаються публічними (<code>[AllowAnonymous]`</code> ), оскільки без них неможливо отримати доступ. <code>/api/secureddevices/user-info`</code> захищені (<code>[Authorize]`</code>).

9. Поділ повноважень (ролі)	Ролі створюються через `RoleManager`. Обмеження доступу реалізовано атрибутом `[Authorize(Roles = "Admin,Manager")]` над конкретними методами контролера.
10. Обробка помилок авторизації	Якщо токена немає або він невалідний, сервер повертає `401 Unauthorized`. Якщо токен є, але роль не підходить (недостатньо прав) - повертається `403 Forbidden`.
11. Аутентифікація vs Авторизація	Аутентифікація відповідає на питання "Хто ти?" (перевірка логіна/пароля і видача токена). Авторизація відповідає на питання "Що тобі дозволено робити?" (перевірка ролей і прав за допомогою `[Authorize]`). Вони працюють послідовно.